

Contents

1	テーブル定義書 (メイン)	2
1.1	1. 文書概要	2
1.1.1	1.1 目的	2
1.1.2	1.2 対象範囲	2
1.1.3	1.3 版数	2
1.1.4	1.4 関連ドキュメント	2
1.2	2. テーブル一覧	2
1.2.1	2.1 retention_class 値の正本定義 (audit_logs.retention_class)	4
1.2.2	2.2 主要関係性 (§ 7.3 から移管)	5
1.2.3	2.3 ユーザー権限保持方式 (3 表モデル)	5
1.3	3. テーブル詳細	6
1.3.1	3.1 users (ユーザーマスタ、全ロール共通の認証情報)	6
1.3.2	3.1a contract_owners (契約者ユーザー、オーナー専有属性)	7
1.3.3	3.2 project_users (プロジェクトユーザー割当 + プロジェクト別ロール)	8
1.3.4	3.4 sessions (セッション)	9
1.3.5	3.5 access_tokens (アクセストークン)	9
1.3.6	3.6 projects (プロジェクト)	9
1.3.7	3.7 project_legacy_keys (レガシー API キー)	10
1.3.8	3.8 allowed_domains (許可ドメイン)	10
1.3.9	3.8a project_ip_allowlist (プロジェクト IP 許可リスト)	10
1.3.10	3.9 faqs (FAQ)	11
1.3.11	3.10 faq_revisions (FAQ 改訂履歴)	11
1.3.12	3.11 faq_search_fts (FAQ 全文検索、FTS5 仮想テーブル)	12
1.3.13	3.12 question_logs (質問ログ)	12
1.3.14	3.13 inquiries (案件)	13
1.3.15	3.14 inquiry_status_history (案件状態履歴)	13
1.3.16	3.15 inquiry_contacts (エンドユーザー連絡先、PII 暗号化)	14
1.3.17	3.16 chat_rooms (個別チャット部屋)	14
1.3.18	3.16b chat_room_reminder_history (チャット部屋リマインド遷移履歴)	14
1.3.19	3.17 chat_messages (チャットメッセージ)	15
1.3.20	3.18 notification_logs (通知ログ)	15
1.3.21	3.19 ai_threshold_persistent_cache (AI しきい値キャッシュ)	16
1.3.22	3.20 email_suppression_list (メールサプレッションリスト、全契約横断)	16
1.3.23	3.21 audit_logs (監査ログ、メイン側 API 操作ログ)	16
1.3.24	3.22 error_logs (エラーログ)	17
1.3.25	3.23 billing_subscriptions (課金サブスクリプション)	17
1.3.26	3.24 billing_invoices (請求書、7 年保持)	18
1.3.27	3.25 usage_metering (利用量計測)	18
1.3.28	3.26 owner_quota_overrides (契約別レート / 予算上書き、統一実装)	18
1.3.29	3.27 service_announcements (お知らせ Control Plane)	19
1.3.30	3.27b announcement_audiences (お知らせ配信対象 M:N)	19
1.3.31	3.28 announcement_recipients (お知らせ配信先 Control Plane)	20
1.3.32	3.29 inbox_messages (受信箱 Tenant Plane)	20
1.3.33	3.30 withdrawal_requests (退会申請)	20
1.3.34	3.31 terms_versions (規約版数、全契約横断)	20
1.3.35	3.32 terms_agreements (規約同意)	21
1.4	4. ER 図 (主要部分、§ 7.2 から移管)	21
1.5	5. テーブル間関連 (§ 2.2 と同期)	21
1.6	6. 命名規則・型方針 (§ 7.5 から移管)	21
1.7	7. SaaS データ分離観点	22
1.7.1	7.1 オーナー境界によるデータ分離方式 (§ 2.6 + § 7.1 から移管)	22
1.7.2	7.2 オーナー / プロジェクト管理者 / メンバーの判定	22
1.7.3	7.3 全契約横断テーブル (contract_owner_user_id を持たない)	22
1.8	8. 状態コード値・状態遷移 (§ 4.x から移管)	22
1.8.1	8.1 FAQ 状態遷移 (faqs.status)	22
1.8.2	8.2 案件状態遷移 (inquiries.case_status、共有概念正本)	23
1.8.3	8.3 チャット部屋自動クローズ 6 段階 (chat_rooms.reminder_state)	23
1.8.4	8.4 通知状態遷移 (notification_logs.delivery_state)	23
1.8.5	8.5 契約状態遷移 (contract_owners.contract_status、共有概念正本)	23

1.9	9. データ削除モード	24
1.9.1	9.1 削除モード対応表	24
1.9.2	9.2 論理削除の物理削除バッチ (DD14 正本)	25
1.10	10. データ保持の調整制約 (NFR-702)	25
1.11	11. PII 取扱 (§ 7.10 から移管)	25
1.12	12. インデックス方針 (§ 7.6 から移管)	25
1.13	13. 未決事項	26

1 テーブル定義書 (メイン)

1.1 1. 文書概要

1.1.1 1.1 目的

メインシステムのデータベース (Cloudflare D1) に存在するすべてのテーブルについて、カラム / 型 / 制約 / インデックス / 外部キー / コード値 / 保持期間 / 保持起点 / **contract_owner_user_id** 分離方式を一元化する。

1.1.2 1.2 対象範囲

- 対象: 利用者向けデータ 32 テーブル (認証 / プロジェクト / FAQ / 案件 / チャット / 通知 / 課金 / お知らせ / 監査 / 規約 等)
- 対象外: 運営者側専用テーブル (**operator_***, **audit_logs**(ハッシュチェーン構造の運営者部分), **web-hook_events**, **operator_approvals**, **accounts_retired** 等)—— 02_ 運営者システム/個別設計書群/04_ テーブル定義書.md 参照

1.1.3 1.3 版数

項目	値
版数	1.0
更新日	2026-05-17

1.1.4 1.4 関連ドキュメント

ドキュメント名	役割	参照先
索引	11 ドキュメント体系の俯瞰	00_ 索引.md
共有概念対応表	contract_owners.contract_status / オーナーユーザーモデルなど	../共有/共有概念.md
API 設計書	テーブルを操作する API	03_API 設計書.md
認証・認可設計書	オーナー境界判定	09_ 認証認可設計書.md
権限設計書	project_users.role(3 ロール定義)	05_ 権限設計書.md
セキュリティ設計書	PII 暗号化 / 監査 / 削除モード	10_ セキュリティ設計書.md
課金・請求設計書	contract_owners.contract_status 正本 / 契約上書きテーブル	11_ 課金請求設計書.md
詳細設計書	実装関連の詳細 (モジュール構成 / バッチ / マイグレーション)	../03_ 詳細設計書.md

1.2 2. テーブル一覧

#	テーブル名	論理名	概要	主キー	contract_owner_u	主管 FR	保持期間	保持起点	retention_class
1	users	ユーザーマスタ	オーナー / プロジェクト管理者 / メンバー全員の認証情報を一元保持 (contract_owners 行存在でオーナー判定)	id	(users 単独では持たず、contract_owners.user_id / project_users.contract_owner_user_id で表現)	FR-001, FR-015	永久 (契約存続中)/ 退会後は § 7 削除ポリシー	作成日時	general

#	テーブル名	論理名	概要	主キー	contract_owner_user_id	主管FR	保持期間	保持起点	retention_class
1a	contract	契約者ユーザー	オーナー専有属性 (contract_status / data_deletion_mode)。	user_id	(本テーブル自身が契約境界、user_id = contract_owner_user_id)	FR-015a, FR-333	永久(契約存続中)/ 退会後は \$ 7 削除ポリシー	作成日時	billing
2	project	プロジェクトユーザー割当 + プロジェクト別ロール	オーナー/メンバーのプロジェクトアクセスとロール (admin / member)	(user_id, project_id)	あり (contract_owner_user_id で契約境界冗長保持)	FR-015, FR-030-035	紐づくユーザーに従属	-	general
4	session	セッション	セッション管理 (複数デバイス対応)	id	(users 経由)	FR-022, FR-332	期限切れ後 90 日	最終アクティビティ	general
5	access_token	トークン	招待 / パスワード再設定 / 再入室等のトークン	id	(users 経由)	FR-004, FR-008	使用后 30 日	発行日時	general
6	projects	プロジェクト	FAQ プロジェクト / ウィジェット設定	id	あり	FR-030-035	永久(プロジェクト存続中)	作成日時	general
7	project_legacy_key	Legacy Key	ウィジェット ローテーション 30 日 猶予	id	(projects 経由)	-	30 日	ローテーション日時	general
8	allowed_domains	許可ドメイン	ウィジェット埋め込み許可ドメイン	id	(projects 経由)	FR-150-156	プロジェクトに従属	-	general
8a	project_allowed_ip_list	プロジェクト IP 許可リスト	FAQ ウィジェット向け IP 制限 (プロジェクト単位 / FR-179・FR-330)	id	(projects 経由)	FR-179, FR-330	プロジェクトに従属	-	general
9	faqs	FAQ	FAQ 本体	id	あり	FR-040-048, FR-322	永久 (FAQ 存続中)	作成日時	general
10	faq_revision	FAQs 改訂履歴	全文スナップショット 50 件保持	id	(faqs 経由)	FR-322	永久 (版ごと)	改訂日時	general
11	faq_search	FAQ 全文検索	FTS5 仮想テーブル (trigram)	(rowid)	(faqs 連動)	FR-300	faqs に従属	-	-
12	question_log	質問ログ	エンドユーザーからの質問 / AI 推論	id	あり	FR-003, NFR-601	730 日 (自動匿名化)	投稿日時	general
13	question_log_faq_refs	参照 FAQ	質問と FAQ の M:N 関係	id	(経由)	-	question_logs に従属	-	general
14	inquiries	案件	未解決質問のグループ	id	あり	FR-070-079	永久 (case_status=closed 後は NFR-706)	作成日時 / closed 日時	general
15	inquiry_status_history	案件状態履歴	状態変更トレース	id	(inquiries 経由)	FR-077	inquiries に従属	-	general
16	inquiry_contact	案件連絡先	エンドユーザーメール (AES-256-GCM 暗号化 + HMAC)	id	(inquiries 経由)	FR-079, NFR-319	inquiries に従属 (物理削除 / 匿名化)	-	general

#	テーブル名	論理名	概要	主キー	contract_owner_u	主管FR	保持期間	保持起点	retention_class
17	chat_rooms	個別チャット部屋	エンドユーザーとの個別チャット(自動クローズ6段階)	id	(inquiries 経由)	FR-080-091	730 日(自動匿名化)	クローズ日時と最終投稿日時の遅い方	general
17	chat_rooms	チャット部屋履歴	6段階自動クローズの状態遷移トレース	id	(chat_rooms 経由)	FR-080-091	chat_rooms に従属	-	general
18	chat_messages	メッセージ	個別チャットの投稿	id	(chat_rooms 経由)	FR-080-091	chat_rooms に従属	-	general
19	notification_logs	通知ログ	メール通知の送信履歴	id	あり	NFR-506, NFR-319	1年	送信日時	general
20	ai_threats	AI脅威検出履歴	3段階しきい値の永続キャッシュ	id	あり(scope=owner 時)	FR-341	30 日	更新日時	general
21	email_suppressions	サブスクリプション	パウンス・苦情の全契約横断管理	id	(全契約横断)	FR-149b	永久	サブレス日時	general
22	audit_logs	監査ログ(メイン側、API 操作ログ専用)	メイン管理者向けの API 操作ログ。ハッシュチェーン正本は運営者側 audit_logs(IF #8 で片方向同期)	id	あり(contract_owner_user_id カラム)	NFR-602a	retention_class 制(1/7/5 年)	操作日時	general / billing / operator_high_priv
23	error_logs	エラーログ	サーバーエラー記録	id	あり(可)	NFR-704	180 日	発生日時	general
24	billing_invoices	課金サブスクリプション	Stripe Subscription 連動	id	あり	FR-148	永久(契約継続中)	作成日時	billing
25	billing_requests	請求書	月次請求書(電子帳簿保存法)	id	あり	FR-148, NFR-602b	7 年	発行日時	billing
26	usage_metrics	利用量計測	質問数/FAQ 件数/チャット部屋数	id	あり	FR-148	7 年	計測日時	billing
27	owner_overrides	契約別 overrides 予算上書き	運営者上書き(SCR-093)——レート/予算ともに本テーブルで統一実装(resource_kind で区別)	id	あり	IF #5	5 年	設定日時	operator_high_priv
28	service_notifications	お知らせ(Control Plane)	運営者発信のお知らせ本体	id	(全契約横断 or 限定)	-	5 年	配信日時	operator_high_priv
28	announcement_targets	お知らせ配信対象 M:N	お知らせの配信先 owner 限定指定(行 0 件 = 全契約配信)	(announcement_id, contract_owner_user_id)	(限定)	-	親お知らせに従属	-	operator_high_priv
29	announcement_recipients	お知らせ受信者	お知らせ配信先・配信集計・監査	id	あり	-	5 年	配信日時	general
30	inbox_messages	受信箱(Tenant Plane)	利用者のお知らせ・通知の既読管理	id	あり	FR-182-192	1 年	既読日時	general
31	withdrawal_requests	退会申請	退会申請レコード(30 日猶予)	id	あり	FR-189	7 年	申請日時	billing
32	terms_versions	規約版数	規約/プライバシーポリシーの版	version	(全契約横断)	-	永久	制定日時	general
33	terms_agreements	規約同意	利用者ごとの同意履歴	id	(users 経由)	-	7 年	同意日時	billing

1.2.1 2.1 retention_class 値の正本定義 (audit_logs.retention_class)

値	保持期間	用途	例
general	1年(NFR-602a)	業務監査	通常の操作 / チャット / FAQ 編集 / ログイン
billing	7年(NFR-602b、電子帳簿保存法)	課金・法定保管	請求 / 支払 / サブスク / 予算 / owner:suspend prefix
operator_high_privilege	5年(NFR-602d、SOX 類似)	運営者高権限操作	復元 / 契約停止 / 物理削除 / AI 変更 / レート上書き / ローテーション

詳細は [10_セキュリティ設計書.md §7](#)(運営者側 §7 が正本) を参照。

1.2.2 2.2 主要関係性 (§7.3 から移管)

関係	型	説明
ユーザー ☒ 契約者	1:0..1	users 1 行に対し contract_owners は 0 行(メンバー)or 1 行(オーナー)。contract_owners 行存在 = オーナー判定
契約者 ☒ プロジェクト	1:N	契約削除時にプロジェクトも削除対象 (projects.contract_owner_user_id REFERENCES contract_owners(user_id))
プロジェクト ☒ FAQ	1:N	FAQ はプロジェクトに帰属
プロジェクト ☒ 未解決質問	1:N	-
未解決質問 ☒ チャット部屋	1:1	-
チャット部屋 ☒ メッセージ	1:N	-
未解決質問 ☒ FAQ	0..1:N	faqs.source_unresolved_question_id で参照可
お知らせ ☒ 受信箱	1:N	service_announcements → announcement_recipients → inbox_messages の fan-out

1.2.3 2.3 ユーザー権限保持方式 (3 表モデル)

採用方式は **3 表分離型**(users + contract_owners + project_users):

- **users**: 全ロール共通の認証主体(認証情報 + PII)。「オーナーかどうか」は **users** 単独では判定せず、**contract_owners** 行存在で判定する
- **contract_owners**: オーナー専有属性 (**contract_status** / **data_deletion_mode**) を分離。**user_id** PRIMARY KEY により 1 契約 = 1 オーナー固定が DB 制約で担保される
- **project_users**: プロジェクト単位ロール (**admin** / **member**)。(**user_id**, **project_id**) 複合 PK + **contract_owner_user_id** 冗長保持(認可境界クエリ高速化と意図明示) + **granted_at** / **granted_by** 保持で行レベル監査

オーナーも **project_users** に行を持つ(プロジェクト作成時に **project_users**(**user_id**= オーナー, **project_id**= 新 PJ, **contract_owner_user_id**= オーナー, **role**='admin', **valid**=1) を自動 INSERT。FR-030a)。認可フロー先頭の **isOwner=true** **bypass**(**contract_owners** 行存在で判定) は維持し、**project_users** 行は critical 通知宛先解決と画面表示(プロジェクト管理者一覧)の事実情報として参照する。個別権限フラグ方式は採用しない(UI / 認可ロジック双方の複雑さに見合わない)。

最後の管理者保護判定(E-AUTHZ-LAST-ADMIN-PROTECTED): オーナー由来の **project_users** 行を判定対象から除外する。具体的には **pu.user_id NOT IN (SELECT user_id FROM contract_owners WHERE valid=1)** で「メンバー由来 admin 行」のみを抽出して件数判定する(オーナーは常に admin 行を持つため判定対象外)。

valid カラムによる論理削除を全削除対象テーブルに適用する(対象テーブルは §9 削除モード表を正本)。**valid** INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)) を追加し、論理削除は UPDATE ... SET **valid**=0 で表現する。論理削除データは **updated_at** 基準で 90 日経過後に物理削除バッチで物理削除する(DD14_バッチ・非同期処理.md を正本)。**deleted_at** カラムは持たず、論理削除日時は既存 **updated_at**(**valid** が 0 に更新された時刻) で代用する(退会日時の証跡は **withdrawal_requests.applied_at**、業務クローズ日時の **inquiries.closed_at** は別概念として維持)。例外テーブル: **audit_logs**(メイン側は API 操作ログ専用、**created_at** + **retention** 期間で物理 DELETE。ハッシュチェーン正本は運営者側)、**sessions**(**revoked_at** で代用)、法定保管 7 年テーブル(**billing_invoices** / **usage_metering** / **withdrawal_requests** / **terms_versions** / **terms_agreements**)、全契約横断テーブル(**email_suppression_list**)、ログ系(**notification_logs** / **error_logs** / **inbox_messages**)、トークン / 履歴系(**access_tokens** / **faq_revisions** / **faq_search_fts** / **chat_messages** / **inquiry_status_history** / **question_log_faq_refs** / **chat_room_reminder_history**)、M:N 関係系(**announcement_audiences**)。

1.3 3. テーブル詳細

1.3.1 3.1 users(ユーザーマスタ、全ロール共通の認証情報)

項目	内容
テーブル名	users
論理名	ユーザーマスタ
概要	オーナー / プロジェクト管理者 / メンバー全員の認証情報 (メール / パスワード / 氏名 / セッション状態など) を一元保持
主キー	id(TEXT, ULID)
設計判断	「オーナーかどうか」は本テーブル単独では判定せず、contract_owners 行存在で判定する (is_owner カラムも owner_account_id 自己参照も持たない)。「契約所属」は contract_owners.user_id = users.id (本人がオーナー) または project_users.user_id = users.id 経由で取得する contract_owner_user_id で表現する。論理削除は valid カラムで表現 (valid=0 = 論理削除済、valid=1 = 有効)。deleted_at 列は持たず、論理削除日時は updated_at で代用する。email_key_version は ローテーション dual-decrypt 期間中の バージョン参照用

1.3.1.1 DDL

```
CREATE TABLE users (  
  id TEXT PRIMARY KEY, -- ULID  
  email_encrypted TEXT NOT NULL,  
  email_hmac TEXT NOT NULL,  
  email_key_version INTEGER NOT NULL DEFAULT 1, -- ローテーション  
  password_hash TEXT,  
  name TEXT,  
  role TEXT NOT NULL CHECK (role IN ('admin', 'end_user')), -- 'service_op  
  status TEXT NOT NULL DEFAULT 'pending_verification'  
  CHECK (status IN ('pending_verification', 'pending_activation', 'active')),  
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),  
  last_login_at TEXT,  
  email_verified_at TEXT,  
  created_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);  
CREATE INDEX idx_users_email_hmac ON users(email_hmac);  
CREATE INDEX idx_users_valid ON users(valid) WHERE valid = 0;
```

1.3.1.2 「契約内メール一意」UNIQUE の再設計 (users 単独では表現不可) 旧 uq_accounts_owner_email(owner_account_id, email_hmac) WHERE valid=1 の代替は users 単独では表現不可。アプリ層トランザクションで次のように担保する:

- ・オーナー登録時: 単に新 users 行 INSERT (契約はまだ無いので重複なし)
- ・メンバー招待時: BEGIN IMMEDIATE で「同一 contract_owner_user_id 配下の既存 project_users.user_id の users.email_hmac セット」+「contract_owners 行の users.email_hmac」を SELECT し重複検査 → 新 users 行 INSERT → project_users 行 INSERT

D1 はトリガによる強制も可能だが MVP では単純化のためアプリ層実装で十分 (契約内ユーザー数は数十オーダー、招待は低頻度)。

1.3.1.3 コード値: role / status

カラム	値	説明
role	admin	利用者管理者 (オーナー / プロジェクト管理者 / メンバーの認証主体ロール、契約境界内)
role	end_user	エンドユーザー (ウィジェット利用者)
role	service_operator	CHECK 制約には含めない。運営者は別 D1 (運営者システム service_operators テーブル) で管理する。メイン側 D1 に運営者 ID は格納しない。audit_logs.actor_role の許容値としては利用する (運営者によるメイン API 呼出口ログを記録するための別概念)
status	pending_verification	メール未確認
status	pending_activation	招待中 (初回パスワード未設定)

カラム	値	説明
status	active	有効
valid	1	有効
valid	0	論理削除済み(論理削除日時は updated_at で判別。updated_at < now() - 90d AND valid=0 の行は物理削除バッチで物理削除される)

1.3.2 3.1a contract_owners(契約者ユーザー、オーナー専有属性)

項目	内容
テーブル名	contract_owners
論理名	契約者ユーザー(オーナー専有属性)
概要	オーナー専有属性(contract_status / data_deletion_mode)を分離。行存在 = オーナー判定(is_owner カラム不要)
主キー	user_id(users(id)への1:1参照)
設計判断	user_id PRIMARY KEY により1契約=1オーナー固定がDB制約で担保される。「契約を持たないオーナー」のような不整合状態は表現不能。contract_status='deleted' ⇔ valid=0の同期はCHECKで強制

1.3.2.1 DDL

```

CREATE TABLE contract_owners (
  user_id          TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  contract_status  TEXT NOT NULL DEFAULT 'active'
                    CHECK (contract_status IN ('active', 'suspended', 'deleted_pending', 'deleted')),
  data_deletion_mode TEXT NOT NULL DEFAULT 'physical'
                    CHECK (data_deletion_mode IN ('physical', 'anonymize')),
  valid            INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at       TEXT NOT NULL,
  updated_at       TEXT NOT NULL,
  -- contract_status='deleted' のとき必ず valid=0(両者は同時セット)
  CHECK ((contract_status <> 'deleted' AND valid = 1)
        OR (contract_status = 'deleted' AND valid = 0))
);
CREATE INDEX idx_contract_owners_status ON contract_owners(contract_status);
CREATE INDEX idx_contract_owners_valid  ON contract_owners(valid) WHERE valid = 0;

```

1.3.2.2 コード値: contract_status / data_deletion_mode

カラム	値	説明
contract_status	active	有効(共有概念正本)
contract_status	suspended	サスペンション中
contract_status	deleted_pending	退会申請中(30日猶予)
contract_status	deleted	削除済み(オーナー退会完了。valid=0も同時セット=CHECK制約で強制)
data_deletion_mode	physical	物理削除(初期値)
data_deletion_mode	anonymize	匿名化

詳細状態遷移は [10_課金・請求設計.md §4](#) を正本とする。

1.3.2.3 contract_status × valid の整合

contract_status	valid	状態
active / suspended	1	オーナー稼働中
deleted_pending	1	オーナー退会申請中(30日猶予)
deleted	0	オーナー退会完了(valid=0と必ず同時セット=CHECK)

メンバーの論理削除は本テーブルではなく **users.valid=0** で表現する(**contract_owners** 行を持たないため)。

1.3.3 3.2 project_users(プロジェクトユーザー割当 + プロジェクト別ロール)

オーナー / プロジェクト管理者 / メンバーの全員に対し、プロジェクトごとに **admin**(プロジェクト管理者) / **member**(メンバー) のロールを付与する。オーナーも本テーブルに行を持つ(プロジェクト作成時に (**user_id**= オーナー, **project_id**= 新 PJ, **contract_owner_user_id**= オーナー, **role**='admin', **valid**=1) 行を自動 INSERT)。認可フロー先頭の **isOwner=true** **bypass(contract_owners** 行存在で判定) は維持し、本テーブルは通知宛先解決と画面表示に使う。

割当ゼロ状態の発生条件: メンバー削除は **ユーザー論理削除** (**UPDATE users SET valid=0**) で行い、**project_users** 行のみの単独削除(離脱)操作は提供しない。論理削除に伴い **project_users** 行も **valid=0** に同時 UPDATE する。**valid=1** 行が 0 行残った状態は構造的に発生しない(ユーザー論理削除と同時に発生するため)。プロジェクト削除起因で **valid=1** 割当が 0 件になる(かつ非オーナーの)メンバーは同一トランザクション内で **users.valid=0** に論理削除する(下記「プロジェクト削除時の孤立メンバー処理」)。最後の admin の有無に関しては、オーナーが常に admin 行を保持するため「プロジェクトに admin 行が 0 件になる」状態は構造的に発生しない(**E-AUTHZ-LAST-ADMIN-PROTECTED** の判定はオーナー由来 admin 行を **contract_owners** JOIN で除外、メンバー由来の admin のみで判定)。

1.3.3.1 DDL

```
CREATE TABLE project_users (  
  user_id          TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  project_id       TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,  
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,  
  role            TEXT NOT NULL CHECK (role IN ('admin','member')),  
  valid           INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),  
  granted_at      TEXT NOT NULL,  
  granted_by      TEXT REFERENCES users(id) ON DELETE SET NULL, -- 招待者削除でも整合維持  
  updated_at      TEXT NOT NULL,  
  PRIMARY KEY (user_id, project_id)  
);  
-- 同一 (user_id, project_id) ペアで valid=1 行は 1 件のみ、再招待時は valid=0 行を残したまま valid=1  
CREATE INDEX idx_project_users_project      ON project_users(project_id);  
CREATE INDEX idx_project_users_owner        ON project_users(contract_owner_user_id);  
-- プロジェクト管理者の高速検索用 (critical 通知宛先解決でも使用、valid=1 の admin のみ)  
CREATE INDEX idx_project_users_role_admin  
  ON project_users(user_id) WHERE role = 'admin' AND valid = 1;  
CREATE INDEX idx_project_users_valid        ON project_users(valid) WHERE valid = 0;
```

PK と再招待: (**user_id, project_id**) の物理 PK は維持。同じユーザー × 同じプロジェクトの再招待は、既存行の **valid** を 0 → 1 に切り替える UPSERT(**INSERT ... ON CONFLICT(user_id, project_id) DO UPDATE SET valid=1, role=?, granted_at=now(), granted_by=?, updated_at=now()**) で表現する。これにより、同一ペアで **valid=1** 行は最大 1 件となり、部分 **UNIQUE** インデックスは不要。論理削除済み (**valid=0**) 行の **granted_at** は最初の招待時刻を維持し、再招待時に上書きしない仕様としても良いが、MVP では復活時に最新値で上書きする(監査ログ側で履歴を担保)。

1.3.3.2 コード値: role 2 値

値	表示名 (UI)	該当プロジェクト範囲で可能な操作
admin	プロジェクト管理者	FAQ 管理 / 個別チャット対応 / ログ参照 / ユーザー管理(該当プロジェクトへの招待・離脱・ロール変更)
member	メンバー	FAQ 管理 / 個別チャット対応 / ログ参照

詳細は [05_権限設計書.md §3](#) を正本とする。

1.3.3.3 プロジェクト削除時の孤立メンバー処理 (アプリ層、論理削除) プロジェクト削除と孤立メンバー cleanup は論理削除で統一する。次の処理はアプリ層のトランザクション内で明示する:

1. **UPDATE project_users SET valid=0, updated_at=now() WHERE project_id=? AND valid=1**(オーナー自身の admin 行も含む全行)

2. UPDATE 後に「他プロジェクト割当 (**valid=1** 行) が 0 件かつ **contract_owners** 行を持たない (非オーナー)」となったメンバー **users** 行を抽出して UPDATE **users** SET **valid=0**, **updated_at=now()** WHERE **id IN (...)** および全セッション失効 (UPDATE **sessions** SET **revoked_at=now()**)
3. UPDATE **projects** SET **status='deleted'**, **valid=0**, **updated_at=now()** WHERE **id=?** (**deleted_at** 列は持たず、**updated_at** で削除日時を代用)
4. 関連テーブル (**allowed_domains** / **project_ip_allowlist** / **faqs** / **inquiries** / **inquiry_contacts** / **chat_rooms** / **question_logs**) の対象行を **valid=0** に伝播

オーナーは **users** 本体の論理削除対象外 (**contract_owners** 行を持つユーザーは常に **valid=1** 維持)。当該プロジェクトに対する **project_users** 行は **valid=0** に論理削除する。論理削除データは **updated_at < now() - 90d** AND **valid=0** の物理削除バッチ (DD14) で 90 日後に物理削除される。物理削除時に **users** の物理 DELETE が走ると ON DELETE CASCADE で関連行も連鎖物理削除されることに留意。詳細は [DD02_プロジェクト・FAQ 管理.md](#) を参照。

1.3.4 3.4 sessions(セッション)

1.3.4.1 DDL(行 5894-5905)

```
CREATE TABLE sessions (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  ip_address  TEXT NOT NULL,
  user_agent  TEXT,
  created_at  TEXT NOT NULL,
  last_accessed_at TEXT NOT NULL,
  expires_at  TEXT NOT NULL,
  revoked_at  TEXT
);
CREATE INDEX idx_sessions_user ON sessions(user_id) WHERE revoked_at IS NULL;
CREATE INDEX idx_sessions_expires ON sessions(expires_at) WHERE revoked_at IS NULL;
```

1.3.5 3.5 access_tokens(アクセストークン)

1.3.5.1 DDL(行 5908-5920)

```
CREATE TABLE access_tokens (
  id          TEXT PRIMARY KEY,
  user_id     TEXT REFERENCES users(id) ON DELETE CASCADE,
  token_hash  TEXT NOT NULL,
  purpose     TEXT NOT NULL CHECK (purpose IN ('email_verify', 'password_reset', 'activation', 'reentry')),
  meta        TEXT,
  created_at  TEXT NOT NULL,
  expires_at  TEXT NOT NULL,
  used_at     TEXT
);
CREATE UNIQUE INDEX uq_access_tokens_hash ON access_tokens(token_hash);
CREATE INDEX idx_access_tokens_expires ON access_tokens(expires_at) WHERE used_at IS NULL;
```

1.3.5.2 コード値: purpose

値	用途
email_verify	メール確認
password_reset	パスワード再設定
activation	招待受諾 (初回パスワード設定)
reentry	エンドユーザー再入室

1.3.6 3.6 projects(プロジェクト)

1.3.6.1 DDL(行 5923-5944)

```
CREATE TABLE projects (
  id          TEXT PRIMARY KEY,
```

```

contract_owner_user_id    TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE C
name                      TEXT NOT NULL CHECK (length(name) BETWEEN 1 AND 100),
description               TEXT,
status                   TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active','de
widget_public_key         TEXT NOT NULL UNIQUE,
widget_public_key_expires_at TEXT NOT NULL,
contact_email             TEXT,
contact_email_verified_at TEXT,
settings                 TEXT NOT NULL DEFAULT '{}',
valid                     INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
created_at                TEXT NOT NULL,
updated_at                TEXT NOT NULL,
CHECK (contact_email_verified_at IS NULL OR contact_email IS NOT NULL),
-- status='deleted' のとき必ず valid=0(両者は同時セット)
CHECK ((status = 'active' AND valid = 1) OR (status = 'deleted' AND valid = 0))
);
CREATE INDEX idx_projects_owner_status ON projects(contract_owner_user_id, status);
CREATE INDEX idx_projects_widget_key   ON projects(widget_public_key);
CREATE INDEX idx_projects_valid        ON projects(valid) WHERE valid = 0;

```

1.3.7 3.7 project_legacy_keys(レガシー API キー)

1.3.7.1 DDL(行 5947-5954)

```

CREATE TABLE project_legacy_keys (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  public_key  TEXT NOT NULL,
  grace_until TEXT NOT NULL,
  created_at  TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_legacy_keys_key ON project_legacy_keys(public_key);

```

1.3.8 3.8 allowed_domains(許可ドメイン)

1.3.8.1 DDL(行 5957-5963)

```

CREATE TABLE allowed_domains (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  domain      TEXT NOT NULL,
  valid       INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL DEFAULT (datetime('now'))
);
-- valid=1 行のみ UNIQUE(再追加可能にするため部分 UNIQUE 化)
CREATE UNIQUE INDEX uq_allowed_domains_project_domain ON allowed_domains(project_id, domain) WH
CREATE INDEX idx_allowed_domains_valid ON allowed_domains(valid) WHERE valid = 0;

```

1.3.9 3.8a project_ip_allowlist(プロジェクト IP 許可リスト)

FAQ ウィジェット (エンドユーザーアクセス) に対する IP 制限を CIDR で保持する。プロジェクト単位設定。空 = 制限なし。管理画面 API は本テーブルの参照対象外 (FR-179 / FR-330)。

1.3.9.1 DDL

```

CREATE TABLE project_ip_allowlist (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  cidr        TEXT NOT NULL,
  family      TEXT NOT NULL CHECK (family IN ('ipv4','ipv6')),
  note        TEXT,
  valid       INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
-- IPv4 / IPv6
-- 評価高速化用に
-- 任意メモ (SCR

```

```

    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now'))
);
-- valid=1 行のみ UNIQUE(再追加可能にするため部分 UNIQUE 化)
CREATE UNIQUE INDEX uq_project_ip_allowlist_project_cidr ON project_ip_allowlist(project_id, ci
CREATE INDEX idx_project_ip_allowlist_project_family ON project_ip_allowlist(project_id, fam
CREATE INDEX idx_project_ip_allowlist_valid ON project_ip_allowlist(valid) WHERE v

```

1.3.9.2 制約・評価

- ・1プロジェクトあたり最大100行(アプリ層で検証)
- ・同一プロジェクト内の重複 CIDR は UNIQUE 制約で拒否
- ・ウィジェット API ハンドラは (project_id, family) インデックスを用いてキャッシュ生成し、判定はメモリ上の LPM(longest-prefix-match) で5分 TTL キャッシュ
- ・監査ログ: audit_logs.action に project.ip_allowlist.update を記録(変更前後の CIDR セット差分を metadata に格納)

1.3.10 3.9 faqs(FAQ)

1.3.10.1 DDL(行 5966-5987)

```

CREATE TABLE faqs (
  id TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),
  body TEXT NOT NULL CHECK (length(body) BETWEEN 1 AND 10000),
  category TEXT,
  status TEXT NOT NULL DEFAULT 'draft'
  CHECK (status IN ('draft', 'published', 'hidden', 'deleted')),
  version INTEGER NOT NULL DEFAULT 1,
  tags TEXT, -- JSON 配列 ( `["tag1","tag2"]` )、最大 10 要素、各タグ
  source_unresolved_question_id TEXT REFERENCES inquiries(id) ON DELETE SET NULL,
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_by TEXT REFERENCES users(id) ON DELETE SET NULL, -- 作成者削除
  updated_by TEXT REFERENCES users(id) ON DELETE SET NULL, -- 同上
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
CREATE INDEX idx_faqs_owner_project ON faqs(contract_owner_user_id, project_id);
CREATE INDEX idx_faqs_project_status ON faqs(project_id, status);
CREATE INDEX idx_faqs_source_inquiry ON faqs(source_unresolved_question_id);
CREATE INDEX idx_faqs_category ON faqs(project_id, category) WHERE category IS NOT NULL
CREATE INDEX idx_faqs_valid ON faqs(valid) WHERE valid = 0;

```

1.3.10.2 コード値: status(状態遷移は §8 参照)

値	説明
draft	下書き
published	公開中
hidden	非公開
deleted	論理削除 (valid=0 と同時セット。復元は 運営者のみ = IF #4)

valid は全テーブル共通の論理削除フラグ(status='deleted' のとき必ず valid=0、それ以外は valid=1)。プロジェクト削除に伴うカスケード論理削除では status を変えずに valid=0 のみセットすることがあるが、UI/API では valid=0 の行を常に除外するため利用者には見えない。

1.3.11 3.10 faq_revisions(FAQ 改訂履歴)

1.3.11.1 DDL(行 5992-6008)

```

CREATE TABLE faq_revisions (
  id TEXT PRIMARY KEY,
  faq_id TEXT NOT NULL REFERENCES faqs(id) ON DELETE CASCADE,
  version INTEGER NOT NULL,
  title TEXT NOT NULL,
  body TEXT NOT NULL,
  category TEXT,
  tags TEXT,
  status_snapshot TEXT NOT NULL CHECK (status_snapshot IN ('draft','published','hidden','delete')),
  source TEXT NOT NULL DEFAULT 'manual' CHECK (source IN ('manual','import')),
  created_at TEXT NOT NULL,
  created_by TEXT NOT NULL REFERENCES users(id)
);
CREATE INDEX idx_faq_revisions_faq ON faq_revisions(faq_id, version DESC);
CREATE INDEX idx_faq_revisions_source ON faq_revisions(source);

```

1.3.12 3.11 faq_search_fts(FAQ 全文検索、FTS5 仮想テーブル)

1.3.12.1 DDL(行 6011-6017)

```

CREATE VIRTUAL TABLE faq_search_fts USING fts5(
  title, body, category, tags,
  content='faqs',
  content_rowid='rowid',
  tokenize='trigram'
);

-- FTS5 contentless 同期トリガ (基本設計が正本)
CREATE TRIGGER faq_search_fts_ai AFTER INSERT ON faqs BEGIN
  INSERT INTO faq_search_fts(rowid, title, body, category, tags)
  VALUES (new.rowid, new.title, new.body, new.category, new.tags);
END;
CREATE TRIGGER faq_search_fts_ad AFTER DELETE ON faqs BEGIN
  INSERT INTO faq_search_fts(faq_search_fts, rowid, title, body, category, tags)
  VALUES ('delete', old.rowid, old.title, old.body, old.category, old.tags);
END;
CREATE TRIGGER faq_search_fts_au AFTER UPDATE ON faqs BEGIN
  INSERT INTO faq_search_fts(faq_search_fts, rowid, title, body, category, tags)
  VALUES ('delete', old.rowid, old.title, old.body, old.category, old.tags);
  INSERT INTO faq_search_fts(rowid, title, body, category, tags)
  VALUES (new.rowid, new.title, new.body, new.category, new.tags);
END;

```

トリガの実装手順 / 動作確認は詳細設計 §9.9 を参照。

1.3.13 3.12 question_logs(質問ログ)

1.3.13.1 DDL(行 6020-6042)

```

CREATE TABLE question_logs (
  id TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  user_question TEXT NOT NULL CHECK (length(user_question) BETWEEN 1 AND 2000),
  ai_response TEXT,
  is_resolved INTEGER NOT NULL DEFAULT 0,
  metering_billable INTEGER NOT NULL DEFAULT 0, -- 課金確定時点のイミュータブルなスナップショット
  confidence_score REAL,
  relevance_score REAL,
  ai_model TEXT,
  ai_token_count_input INTEGER,
  ai_token_count_output INTEGER,
  result_type TEXT CHECK (result_type IN ('answered','unanswered','error')),

```

```

result_reason_code    TEXT,
pii_masked            INTEGER NOT NULL DEFAULT 0,
session_id            TEXT,
ip_address             TEXT,
valid                 INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
created_at             TEXT NOT NULL,
updated_at            TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_qlog_owner_created      ON question_logs(contract_owner_user_id, created_at);
CREATE INDEX idx_qlog_project_billable_created ON question_logs(project_id, metering_billable, created_at);
CREATE INDEX idx_qlog_valid              ON question_logs(valid) WHERE valid = 0;

```

1.3.14 3.13 inquiries(案件)

1.3.14.1 DDL(行 6053-6074)—— 詳細設計の **case_status** CHECK 制約欠落を補正

```

CREATE TABLE inquiries (
  id TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  inquiry_code TEXT NOT NULL,
  question_log_id TEXT REFERENCES question_logs(id),
  user_question TEXT NOT NULL,
  case_status TEXT NOT NULL DEFAULT 'open'
  CHECK (case_status IN ('open', 'resolved', 'closed', 'faq_registered')),
  faq_candidate_status TEXT NOT NULL DEFAULT 'none'
  CHECK (faq_candidate_status IN ('none', 'candidate', 'drafted', 'registered')),
  faq_draft TEXT,
  assignee_user_id TEXT REFERENCES users(id) ON DELETE SET NULL,
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  closed_at TEXT
  -- closed_at は業務クローズ日時 (case_status='closed' 時刻) であり、論理削除とは独立した業務概念
);
CREATE INDEX idx_inquiries_contract_status_created ON inquiries(contract_owner_user_id, case_status, created_at);
CREATE INDEX idx_inquiries_project_status ON inquiries(project_id, case_status);
CREATE INDEX idx_inquiries_assignee ON inquiries(assignee_user_id) WHERE case_status = 'open';
CREATE UNIQUE INDEX uq_inquiries_code ON inquiries(inquiry_code);
CREATE INDEX idx_inquiries_valid ON inquiries(valid) WHERE valid = 0;

```

1.3.14.2 コード値: **case_status**(共有概念正本 = FR-079 自動 **closed** 遷移禁止)

値	説明	遷移条件
open	未対応	新規作成時
resolved	解決済み	admin の「解決済み」操作のみ
closed	対応不要終了	admin の明示操作のみ。自動 retention で closed に遷移させない (FR-079)
faq_registered	FAQ 化済み	admin の「FAQ 登録完了」操作

詳細遷移は § 8.2 参照。**case_status** と **valid** は独立した概念で、**closed** は admin による明示クローズ (業務終了)、**valid=0** はプロジェクト削除に伴うカスケード論理削除。両者は別軸として共存する。

1.3.15 3.14 inquiry_status_history(案件状態履歴)

1.3.15.1 DDL(行 6076-6086)

```

CREATE TABLE inquiry_status_history (
  id TEXT PRIMARY KEY,
  inquiry_id TEXT NOT NULL REFERENCES inquiries(id) ON DELETE CASCADE,
  from_status TEXT NOT NULL,
  to_status TEXT NOT NULL,

```

```

    reason          TEXT,
    changed_by      TEXT REFERENCES users(id) ON DELETE SET NULL, -- 操作者削除でも履歴は残す
    changed_at      TEXT NOT NULL
);
CREATE INDEX idx_isth_inquiry_changed ON inquiry_status_history(inquiry_id, changed_at DESC);

```

1.3.16 3.15 inquiry_contacts(エンドユーザー連絡先、PII 暗号化)

1.3.16.1 DDL(行 6089-6100)

```

CREATE TABLE inquiry_contacts (
    id          TEXT PRIMARY KEY,
    inquiry_id  TEXT NOT NULL REFERENCES inquiries(id) ON DELETE CASCADE,
    email_encrypted TEXT,
    email_hmac  TEXT,
    email_key_version INTEGER NOT NULL DEFAULT 1, -- ローテーション dual-decrypt 期間中の バージョン
    display_name TEXT,
    verified    INTEGER NOT NULL DEFAULT 0,
    notify_enabled INTEGER NOT NULL DEFAULT 1,
    valid       INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
    created_at  TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at  TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_inquiry_contacts_email_hmac ON inquiry_contacts(email_hmac);
CREATE UNIQUE INDEX uq_inquiry_contacts_inquiry ON inquiry_contacts(inquiry_id);
CREATE INDEX idx_inquiry_contacts_valid ON inquiry_contacts(valid) WHERE valid = 0;

```

email_encrypted は AES-256-GCM(オーナー派生) で暗号化。email_hmac は HMAC-SHA256 で検索用。
 詳細は [10_セキュリティ設計書.md §3](#) 参照。

1.3.17 3.16 chat_rooms(個別チャット部屋)

1.3.17.1 DDL(行 6103-6121)

```

CREATE TABLE chat_rooms (
    id          TEXT PRIMARY KEY,
    inquiry_id  TEXT NOT NULL REFERENCES inquiries(id) ON DELETE CASCADE,
    room_status TEXT NOT NULL DEFAULT 'open' CHECK (room_status IN ('open', 'closed')),
    reminder_state TEXT NOT NULL DEFAULT 'active'
    CHECK (reminder_state IN ('active', 'stage1_pending_admin', 'stage2_user_no_response', 'stage3_user_no_response', 'stage4_final_no_response', 'stage5_final_no_response', 'stage6_auto_close')),
    last_message_at TEXT,
    last_message_actor_type TEXT,
    reminder_state_changed_at TEXT,
    valid         INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
    created_at    TEXT NOT NULL,
    updated_at    TEXT NOT NULL DEFAULT (datetime('now')),
    closed_at     TEXT
);
CREATE INDEX idx_chat_rooms_inquiry ON chat_rooms(inquiry_id);
CREATE INDEX idx_chat_rooms_open_reminder ON chat_rooms(reminder_state, last_message_at) WHERE reminder_state = 'open';
CREATE INDEX idx_chat_rooms_valid ON chat_rooms(valid) WHERE valid = 0;

```

1.3.17.2 コード値: reminder_state(自動クローズ 6 段階) 詳細は §8.3 参照。

1.3.18 3.16b chat_room_reminder_history(チャット部屋リマインド遷移履歴)

1.3.18.1 DDL

```

CREATE TABLE chat_room_reminder_history (
    id          TEXT PRIMARY KEY,
    chat_room_id TEXT NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,

```

```

from_state    TEXT NOT NULL,
to_state      TEXT NOT NULL CHECK (to_state IN (
    'active', 'stage1_pending_admin', 'stage2_user_check_sent',
    'stage3_user_no_response', 'stage4_final_check',
    'stage5_final_no_response', 'stage6_auto_closed'
)),
reason        TEXT,
changed_by    TEXT REFERENCES users(id) ON DELETE SET NULL,
changed_at    TEXT NOT NULL
);
CREATE INDEX idx_chat_reminder_room_changed
ON chat_room_reminder_history(chat_room_id, changed_at DESC);

```

1.3.18.2 設計判断

- ・ 障害解析・SLA 監査・自動クローズ判定の事後検証で「いつ stage3 に到達したか」を追えるよう、**inquiry_status_history** と対称に履歴テーブルとして配置する
- ・ 状態遷移時はアプリ層で **chat_rooms** 更新と本テーブルへの INSERT を D1 batch API で同一トランザクション化
- ・ **changed_by** は手動遷移(管理者の介入)の場合に記録、自動遷移(リマインドバッチ)では NULL

1.3.19 3.17 chat_messages(チャットメッセージ)

1.3.19.1 DDL(行 6124-6132)

```

CREATE TABLE chat_messages (
    id            TEXT PRIMARY KEY,
    chat_room_id  TEXT NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
    actor_type    TEXT NOT NULL CHECK (actor_type IN ('admin', 'end_user', 'system')),
    actor_id      TEXT REFERENCES users(id) ON DELETE SET NULL,
    content       TEXT NOT NULL CHECK (length(content) BETWEEN 1 AND 2000),
    created_at    TEXT NOT NULL
);
CREATE INDEX idx_chat_messages_room_created ON chat_messages(chat_room_id, created_at);

```

1.3.20 3.18 notification_logs(通知ログ)

1.3.20.1 DDL(行 6241-6260)

```

CREATE TABLE notification_logs (
    id            TEXT PRIMARY KEY,
    contract_owner_user_id  TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
    inquiry_id      TEXT REFERENCES inquiries(id) ON DELETE SET NULL,
    chat_message_id TEXT REFERENCES chat_messages(id) ON DELETE SET NULL,
    recipient_email_hmac TEXT,
    notification_type TEXT NOT NULL,
    delivery_state   TEXT NOT NULL
                    CHECK (delivery_state IN ('queued', 'sending', 'sent', 'delivered',
                    'failed', 'bounced', 'complained', 'suppress

    message_id      TEXT,
    attempt_count    INTEGER NOT NULL DEFAULT 0,
    sent_at          TEXT,
    delivered_at     TEXT,
    failed_at        TEXT,
    fail_reason      TEXT,
    created_at       TEXT NOT NULL
);
CREATE INDEX idx_nlog_owner_created ON notification_logs(contract_owner_user_id, created_at DESC);
CREATE INDEX idx_nlog_message_id   ON notification_logs(message_id);
CREATE INDEX idx_nlog_state        ON notification_logs(delivery_state);

```


1.3.20.2 コード値: **delivery_state**(状態遷移は §8.4 参照)

値	説明
queued	Queue 投入済み
sending	送信中
sent	送信成功
delivered	配信成功 (Resend Webhook 経由)
failed	失敗 (当方再送対象、最大 3 回)
bounced	バウンス (ハードまたはソフト 5 連続)
complained	スパム報告
suppressed	サブプレスリスト追加済み (終了状態)

1.3.21 3.19 **ai_threshold_persistent_cache**(AI しきい値キャッシュ)

1.3.21.1 DDL(行 6323-6335)

```
CREATE TABLE ai_threshold_persistent_cache (  
  id TEXT PRIMARY KEY,  
  contract_owner_user_id TEXT REFERENCES contract_owners(user_id) ON DELETE CASCADE, -- FK 明  
  project_id TEXT REFERENCES projects(id) ON DELETE CASCADE, -- FK 明  
  scope TEXT NOT NULL CHECK (scope IN ('global', 'owner', 'project')),  
  confidence_threshold REAL NOT NULL CHECK (confidence_threshold BETWEEN 0.0 AND 1.0),  
  relevance_threshold REAL NOT NULL CHECK (relevance_threshold BETWEEN 0.0 AND 1.0),  
  version INTEGER NOT NULL,  
  received_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL,  
  -- scope と owner/project の整合 CHECK  
  CHECK (  
    (scope = 'global' AND contract_owner_user_id IS NULL AND project_id IS NULL)  
    OR (scope = 'owner' AND contract_owner_user_id IS NOT NULL AND project_id IS NULL)  
    OR (scope = 'project' AND contract_owner_user_id IS NOT NULL AND project_id IS NOT NULL)  
  );  
CREATE UNIQUE INDEX uq_ai_threshold_scope ON ai_threshold_persistent_cache(scope, COALESCE(cont  
3 階層しきい値: global / owner / project。詳細は 03_API 設計書.md §5.5.5(IF #6) 参照。
```

1.3.22 3.20 **email_suppression_list**(メールサブプレスリスト、全契約横断)

1.3.22.1 DDL(行 6289-6298)

```
CREATE TABLE email_suppression_list (  
  id TEXT PRIMARY KEY,  
  email_hmac TEXT NOT NULL UNIQUE,  
  reason TEXT NOT NULL CHECK (reason IN ('bounced_hard', 'bounced_soft_5x', 'complained', '  
  is_permanent INTEGER NOT NULL DEFAULT 1,  
  created_at TEXT NOT NULL,  
  released_at TEXT  
);  
CREATE INDEX idx_suppression_permanent ON email_suppression_list(is_permanent, email_hmac);  
全契約横断テーブル (R-003 共通ドメインレピュテーション保護対策)。10_セキュリティ設計書.md §11 参照。
```

1.3.23 3.21 **audit_logs**(監査ログ、メイン側 API 操作ログ)

1.3.23.1 DDL(ハッシュチェーン列は持たない)

```
CREATE TABLE audit_logs (  
  id TEXT PRIMARY KEY,  
  contract_owner_user_id TEXT REFERENCES contract_owners(user_id) ON DELETE SET NULL,  
  actor_user_id TEXT REFERENCES users(id) ON DELETE SET NULL,  
  actor_role TEXT CHECK (actor_role IS NULL OR actor_role IN ('admin', 'end_user',
```

```

action                TEXT NOT NULL,
target_type           TEXT,
target_id             TEXT,
ip_address_masked     TEXT,
user_agent            TEXT,
metadata              TEXT, -- JSON 文字列 (差分・補助情報)
retention_class       TEXT NOT NULL DEFAULT 'general'
                     CHECK (retention_class IN ('general','billing','operator_high_priv'));
created_at            TEXT NOT NULL
-- ハッシュチェーン列 (prev_hash / current_hash / tombstone) は持たない。ハッシュチェーン正本は運営者側);
CREATE INDEX idx_audit_owner_action_created ON audit_logs(contract_owner_user_id, action, created_at);
CREATE INDEX idx_audit_retention_created ON audit_logs(retention_class, created_at);
CREATE INDEX idx_audit_actor_created ON audit_logs(actor_user_id, created_at DESC);

```

- ・役割: メイン側 API 操作ログ。SCR-019 操作ログ画面など、メイン管理者向けの即時参照クエリに低レイテンシで応える
- ・ハッシュチェーン正本は運営者側 (運営者システム audit_logs テーブル)。メイン側のレコードは IF #8 で運営者側へ片方向同期され、運営者側でハッシュチェーン化される
- ・3 区分保持 (general 1y / billing 7y / operator_high_priv 5y)。クラス名表記が正本、期間表記は補助
- ・削除方針: 保持期間経過後は物理 DELETE(updated_at 列を持たないため created_at + retention 期間 で判定)。tombstone 方式は採らない(チェーン非対象のため)
- ・actor_role の service_operator は、運営者によるメイン API 呼出 (IF 経由) を記録する用途で許容値として残す。これは users.role の値とは独立した概念
- ・metadata の JSON 構造例: {"before": {...}, "after": {...}, "ticket_id": "OPS-1234"}

1.3.24 3.22 error_logs(エラーログ)

1.3.24.1 DDL(行 9052-6061、詳細設計内記載)

```

CREATE TABLE error_logs (
  id                TEXT PRIMARY KEY,
  contract_owner_user_id TEXT,
  user_id           TEXT,
  url               TEXT,
  error_type        TEXT,
  stack             TEXT,
  occurred_at       TEXT NOT NULL
);
CREATE INDEX idx_error_logs_owner_occurred ON error_logs(contract_owner_user_id, occurred_at DESC);

```

PII / トークン / カード情報を記録禁止 (FR-114、エンドユーザー入力先頭 100 文字 + ハッシュのみ)。

1.3.25 3.23 billing_subscriptions(課金サブスクリプション)

1.3.25.1 DDL(行 6138-6154)

```

CREATE TABLE billing_subscriptions (
  id                TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
  stripe_subscription_id TEXT UNIQUE,
  status            TEXT NOT NULL
                     CHECK (status IN ('trialing','active','past_due','canceled','unpaid'));
  plan_id           TEXT NOT NULL,
  trial_started_at  TEXT,
  trial_ends_at     TEXT,
  current_period_start TEXT,
  current_period_end TEXT,
  cancel_at         TEXT,
  valid             INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at        TEXT NOT NULL,
  updated_at        TEXT NOT NULL
);
CREATE INDEX idx_billing_subs_owner ON billing_subscriptions(contract_owner_user_id);

```

```
CREATE INDEX idx_billing_subs_status ON billing_subscriptions(status);
CREATE INDEX idx_billing_subs_valid ON billing_subscriptions(valid) WHERE valid = 0;
```

物理削除バッチ除外: billing_subscriptions は valid カラムを持つが、電子帳簿保存法 / 法定保管要件との整合のため、90 日後の物理削除バッチ (DD14) の対象外とする。valid=0 行は永久に保持し、Stripe 連動の取消履歴として参照可能にする。

1.3.26 3.24 billing_invoices(請求書、7 年保持)

1.3.26.1 DDL(行 6156-6174)

```
CREATE TABLE billing_invoices (
  id TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id),
  billing_year_month TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('draft','issued','paid','past_due','',
  amount_total INTEGER NOT NULL,
  amount_tax INTEGER NOT NULL DEFAULT 0,
  currency TEXT NOT NULL DEFAULT 'JPY',
  stripe_invoice_id TEXT UNIQUE,
  pdf_r2_key TEXT,
  issued_at TEXT,
  paid_at TEXT,
  refunded_at TEXT,
  created_at TEXT NOT NULL
);
CREATE UNIQUE INDEX uq_billing_invoices_owner_month ON billing_invoices(contract_owner_user_id,
CREATE INDEX idx_billing_invoices_status ON billing_invoices(status);
```

1.3.27 3.25 usage_metering(利用量計測)

1.3.27.1 DDL(行 6176-6190)

```
CREATE TABLE usage_metering (
  id TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
  billing_year_month TEXT NOT NULL,
  question_count INTEGER NOT NULL DEFAULT 0,
  faq_count_snapshot INTEGER NOT NULL DEFAULT 0,
  chat_room_count INTEGER NOT NULL DEFAULT 0,
  ai_token_input INTEGER NOT NULL DEFAULT 0,
  ai_token_output INTEGER NOT NULL DEFAULT 0,
  ai_cost_yen INTEGER NOT NULL DEFAULT 0,
  finalized_at TEXT,
  updated_at TEXT NOT NULL
);
CREATE UNIQUE INDEX uq_usage_metering_owner_month ON usage_metering(contract_owner_user_id, bil
```

1.3.28 3.26 owner_quota_overrides(契約別レート / 予算上書き、統一実装)

重要: rate_limit_overrides / budget_limit_overrides は別テーブルではなく、本テーブル + resource_kind 値で統一実装される。

1.3.28.1 DDL(行 6337-6350)

```
CREATE TABLE owner_quota_overrides (
  id TEXT PRIMARY KEY,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
  resource_kind TEXT NOT NULL
  CHECK (resource_kind IN ('widget_ask_per_min','widget_inquiry_per_min',
  'email_per_hour','admin_api_per_min')),
  threshold INTEGER NOT NULL,
  window_sec INTEGER NOT NULL,
```

```

valid_until          TEXT NOT NULL,
reason               TEXT,
created_by           TEXT,
created_at           TEXT NOT NULL
);
CREATE INDEX idx_tqo_owner_kind ON owner_quota_overrides(contract_owner_user_id, resource_kind,

```

1.3.28.2 コード値: resource_kind

値	用途
widget_ask_per_min	ウィジェット質問レート
widget_inquiry_per_min	ウィジェット未解決質問送信レート
widget_chat_per_min	ウィジェットチャット投稿レート
email_per_hour	メール送信レート
admin_api_per_min	管理 API レート

詳細は [11_課金請求設計書.md §3](#) / [05_権限設計書.md §7](#) 参照。

1.3.29 3.27 service_announcements(お知らせ Control Plane)

1.3.29.1 DDL(行 6196-6208)

```

CREATE TABLE service_announcements (
  id                TEXT PRIMARY KEY,
  title             TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),
  body_html         TEXT NOT NULL,
  importance        TEXT NOT NULL CHECK (importance IN ('low','normal','high','critical')),
  -- 配信対象は announcement_audiences テーブルで M:N 管理 (行 0 件 = 全契約配信)
  published_at      TEXT NOT NULL,
  retracted_at      TEXT,
  created_by        TEXT REFERENCES users(id) ON DELETE SET NULL, -- 作成者削除でも履歴残す
  created_at        TEXT NOT NULL
);
CREATE INDEX idx_announcements_published ON service_announcements(published_at DESC) WHERE retracted_at IS NULL;

```

importance 4 値は共有概念正本(critical はメール強制送信)。07_メッセージ一覧.md §2.4 参照。

配信対象は **announcement_audiences(§3.27b)** で M:N 管理 する。announcement_audiences 行が 0 件のお知らせは「全契約配信」を意味する。

1.3.30 3.27b announcement_audiences(お知らせ配信対象 M:N)

1.3.30.1 DDL

```

CREATE TABLE announcement_audiences (
  announcement_id   TEXT NOT NULL REFERENCES service_announcements(id) ON DELETE CASCADE,
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,
  created_at        TEXT NOT NULL,
  PRIMARY KEY (announcement_id, contract_owner_user_id)
);
CREATE INDEX idx_announcement_audiences_owner ON announcement_audiences(contract_owner_user_id, announcement_id);

```

1.3.30.2 設計判断

- ・ 配信対象は中間テーブルで M:N 管理する (service_announcements 側に JSON 配列の多値列は持たない)。1NF 整合・JOIN 可能性・150 契約スケール時の性能を担保する
- ・ 行 0 件 = 全契約配信 (contract_owners.contract_status='active' AND valid=1 の動的解決)
- ・ 行 1 件以上 = 限定配信 (列举された contract_owner_user_id のみ)
- ・ 配信ファンアウトは announcement_recipients で user 単位に展開される。本テーブルは「契約単位の対象指定」であり、最終配信先解決は announcement_audiences × project_users で実施

1.3.31 3.28 announcement_recipients(お知らせ配信先 Control Plane)

1.3.31.1 DDL(行 6210-6220)

```
CREATE TABLE announcement_recipients (  
  id TEXT PRIMARY KEY,  
  announcement_id TEXT NOT NULL REFERENCES service_announcements(id) ON DELETE CASCADE,  
  contract_owner_user_id TEXT NOT NULL,  
  user_id TEXT NOT NULL,  
  sent_at TEXT NOT NULL,  
  delivery_status TEXT NOT NULL CHECK (delivery_status IN ('pending','delivered','failed')),  
  read_at TEXT  
);  
CREATE INDEX idx_ar_announcement ON announcement_recipients(announcement_id);  
CREATE INDEX idx_ar_owner_user ON announcement_recipients(contract_owner_user_id, user_id);
```

1.3.32 3.29 inbox_messages(受信箱 Tenant Plane)

1.3.32.1 DDL(行 6222-6239)

```
CREATE TABLE inbox_messages (  
  id TEXT PRIMARY KEY,  
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,  
  recipient_user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  category TEXT NOT NULL CHECK (category IN ('billing','announcement','system')),  
  priority TEXT NOT NULL CHECK (priority IN ('low','normal','high','critical')),  
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),  
  body_html TEXT NOT NULL,  
  source_kind TEXT,  
  source_id TEXT,  
  dedup_key TEXT NOT NULL,  
  read_at TEXT,  
  hidden_at TEXT,  
  created_at TEXT NOT NULL  
);  
CREATE INDEX idx_inbox_owner_user_unread ON inbox_messages(contract_owner_user_id, recipient_user_id);  
CREATE INDEX idx_inbox_dedup ON inbox_messages(contract_owner_user_id, dedup_key, created_at);
```

1.3.32.2 お知らせ二層構成

- **Control Plane**(service_announcements / announcement_recipients): 運営者作成お知らせ本体 + 配信集計・監査
- **Tenant Plane**(inbox_messages): 利用者管理者の受信箱(既読管理)
- **announcement** カテゴリは Control → Tenant の二層経由。billing / system は inbox_messages 直接生成
- 重複生成対策: (contract_owner_user_id, event_kind, dedup_key) で 60 分以内は 1 件に集約

1.3.33 3.30 withdrawal_requests(退会申請)

1.3.33.1 DDL(行 6352-6361)

```
CREATE TABLE withdrawal_requests (  
  id TEXT PRIMARY KEY,  
  contract_owner_user_id TEXT NOT NULL REFERENCES contract_owners(user_id) ON DELETE CASCADE,  
  applied_at TEXT NOT NULL,  
  applied_by TEXT NOT NULL REFERENCES users(id),  
  reason TEXT,  
  scheduled_deletion_at TEXT NOT NULL  
);  
CREATE INDEX idx_withdrawal_owner_scheduled ON withdrawal_requests(contract_owner_user_id, scheduled_deletion_at);
```

1.3.34 3.31 terms_versions(規約版数、全契約横断)

1.3.34.1 DDL(行 6300-6308)

```
CREATE TABLE terms_versions (
  version          TEXT PRIMARY KEY,
  effective_date   TEXT NOT NULL,
  body_html        TEXT NOT NULL,
  diff_summary     TEXT,
  notification_sent_at TEXT,
  consent_deadline_days INTEGER NOT NULL DEFAULT 14,
  created_at       TEXT NOT NULL
);
```

1.3.35 3.32 terms_agreements(規約同意)

1.3.35.1 DDL(行 6310-6317)

```
CREATE TABLE terms_agreements (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  terms_version TEXT NOT NULL REFERENCES terms_versions(version),
  agreed_at   TEXT NOT NULL,
  agreed_ip_masked TEXT
);
CREATE UNIQUE INDEX uq_terms_user_version ON terms_agreements(user_id, terms_version);
```

1.4 4. ER 図 (主要部分、§7.2 から移管)

```
erDiagram
  users ||--o| contract_owners : "owner profile (0..1)"
  contract_owners ||--o{ projects : owns
  contract_owners ||--o| billing_subscriptions : has
  contract_owners ||--o{ inbox_messages : "1-N(契約配下)"
  contract_owners ||--o{ audit_logs : "1-N"
  users ||--o{ project_users : "0..N(全ロール、role: admin/member)"
  users ||--o{ sessions : has
  users ||--o{ access_tokens : has
  users ||--o{ terms_agreements : has
  projects ||--o{ project_users : "1-N(プロジェクト割当)"
  projects ||--o{ faqs : contains
  projects ||--o{ question_logs : records
  projects ||--o{ inquiries : holds
  projects ||--o{ allowed_domains : authorizes
  projects ||--o{ project_legacy_keys : rotates
  faqs ||--o{ faq_revisions : versioned
  faqs }o--o| inquiries : "source_unresolved_question_id(opt)"
  inquiries ||--|| chat_rooms : has
  inquiries ||--o{ inquiry_contacts : has
  inquiries ||--o{ inquiry_status_history : tracks
  chat_rooms ||--o{ chat_messages : contains
  service_announcements ||--o{ announcement_recipients : "fan-out"
  announcement_recipients ||--o| inbox_messages : "copied to"
  billing_subscriptions ||--o{ billing_invoices : generates
  contract_owners ||--o{ usage_metering : metered
  contract_owners ||--o{ owner_quota_overrides : overridden
  terms_versions ||--o{ terms_agreements : agreed
```

1.5 5. テーブル間関連 (§2.2 と同期)

§2.2「主要関係性」を参照。

1.6 6. 命名規則・型方針 (§7.5 から移管)

対象	規則
テーブル	snake_case 複数形
主キー	文字列 ID(ULID)/ *_versions 等の例外あり
外部キー	{table_singular}_id
日時	TEXT(ISO 8601 UTC、NFR-1103)
状態	status または {対象}_status
真偽値	INTEGER 0/1
JSON	TEXT 列に JSON 文字列で保存
暗号化	{column}_encrypted(AES-256-GCM) + {column}_hmac(検索用) の 2 列構成

1.7 7. SaaS データ分離観点

1.7.1 7.1 オーナー境界によるデータ分離方式 (§2.6 + §7.1 から移管)

- 全契約スコープのテーブルに **contract_owner_user_id** カラム必須 (全契約横断テーブルを除く —— §7.3 参照)
- 「契約境界 (owner)」を表す FK 系列 → **contract_owners(user_id)** を参照 (契約境界違反を物理的に弾く)。「操作者・主体 (actor)」系 → **users(id)** を参照
- API / 認可レイヤで境界検証: **principal.contractOwnerId === target.contract_owner_user_id** (認証・認可設計書 09 §5)
- 違反時は **404 偽装** (403 だと相手契約のリソース存在が漏れる)
- 150 契約到達時に D1 シャーディング設計を着手予定 (Future 参照)

1.7.2 7.2 オーナー / プロジェクト管理者 / メンバーの判定

- オーナー判定:** **contract_owners.user_id = users.id** 行が存在するユーザー (行存在 = オーナー)
- プロジェクト管理者:** 非オーナー かつ **project_users.role='admin' AND valid=1** を 1 件以上保持するユーザー
- メンバー:** 非オーナー かつ **project_users.role='member' AND valid=1** のみを保持するユーザー
- 同一ユーザーがオーナー兼任 / 複数プロジェクト所属する場合の権限解決は、認可フロー先頭の **isOwner bypass** を最優先 (08_認証・認可設計.md §5 を正本)
- 1 契約 = 1 オーナー固定は **contract_owners.user_id PRIMARY KEY** で DB レベル担保 (**is_owner** カラムや **owner_account_id** 自己参照は持たない)

1.7.3 7.3 全契約横断テーブル (contract_owner_user_id を持たない)

テーブル名	理由
email_suppression_list	メール bounce / 苦情の全契約横断管理 (R-003 共通ドメイン対策)
terms_versions	規約 / プライバシーポリシーの版 (全利用者共通)
service_announcements	運営者発信のお知らせ本体 (配信先は announcement_recipients で契約別)

1.8 8. 状態コード値・状態遷移 (§4.x から移管)

1.8.1 8.1 FAQ 状態遷移 (faqs.status)

From	To	トリガー	条件
-	draft	FAQ 作成	admin のみ
draft	published	公開操作	admin のみ、interactive セッション、内容確認ガード (基本設計書 §4.2.1)
draft	hidden	非公開保存	admin のみ
published	hidden	非公開化	admin のみ
hidden	published	再公開	admin のみ
any	deleted	論理削除	admin のみ
deleted	draft	復元	運営者のみ (IF #4)

ガード: **draft → published** 直接遷移を 3 層で禁止 (API / Domain / DB)。公開状態への遷移は管理者の明示操作のみ。

1.8.2 8.2 案件状態遷移 (inquiries.case_status、共有概念正本)

From	To	トリガー	条件	通知
-	open	未解決質問登録	システム自動	エンドユーザーへ IN-QUIRY_CREATED
open	resolved	admin 「解決済み」	admin のみ	なし
open	closed	admin 「対応不要終了」	admin の明示操作のみ。自動 retention で closed に遷移させない (FR-079)	エンドユーザーへ in-box(announcement)
any	faq_registered	admin 「FAQ 登録完了」	admin のみ	エンドユーザーへ FAQ_REGISTERED

1.8.3 8.3 チャット部屋自動クローズ 6 段階 (chat_rooms.reminder_state)

From	To	トリガー	経過時間	通知
active	stage1_pending	最終投稿から 30 分経過	7 日	admin へ CHAT_HOLD_CHECK
stage1_pending	stage2_user_check	admin 「利用者待ち」判定 or 14 日未操作	即時 / 14 日	エンドユーザーへ CHAT_RESOLUTION_CHECK
stage2_user_check	stage3_user_no_response	投稿なし	7 日	なし (Cron 同期)
stage3_user_no_response	stage4_final_check	即時遷移	(同周期)	エンドユーザーへ CHAT_FINAL_CONFIRM
stage4_final_check	stage5_final_no_close	投稿なし	7 日	なし
stage5_final_no_close	stage6_auto_close	即時遷移	(3 日後)	エンドユーザーへ CHAT_AUTO_CLOSED、room_status=closed
stage6_auto_close	active (再オープン)	エンドユーザー再アクセス	30 日以内	なし
stage6_auto_close	access_tokens (access_tokens 消滅)	30 日経過	-	エンドユーザーへ「新規部屋案内」
任意の active 系	active	EU / admin 投稿	-	カウンタリセット
active 系	手動 close	admin 明示操作	-	エンドユーザーへ CHAT_CLOSED
closed	active	admin 再オープン	30 日以内	監査ログ記録必須

1.8.4 8.4 通知状態遷移 (notification_logs.delivery_state)

From	To	トリガー
-	queued	Queue 投入
queued	sending	Worker 起動
sending	sent	Resend API 成功
sending	failed	Resend API 失敗 (恒久) —— 当方再送対象 (最大 3 回、NFR-506)
sent	delivered	email.delivered Webhook (終了状態)
sent	bounced	email.bounced Webhook (ハード / ソフト 5 連続)
sent	complained	email.complained Webhook
sent	delayed	email.delivery_delayed Webhook (Resend 再送中)
delayed	delivered / bounced	後続結果
bounced	suppressed	サプレスリスト追加
complained	suppressed	即時永久サプレス (復帰不可)
failed	queued	当方再送 (最大 3 回、3 回失敗 + 24 時間経過で admin へ system 通知)

1.8.5 8.5 契約状態遷移 (contract_owners.contract_status、共有概念正本)

詳細は [11_課金請求設計書.md § 4 / § 5](#) を正本とする。本書では概要のみ:

From	To	トリガー	reason
-	active	契約作成	-
active	suspended	IF #1(支払い失敗 7 日猶予経過)	payment_failure_grace_expired
active	suspended	IF #1(運営者手動)	operator_manual / tos_violation
active	suspended	Cron(トライアル終了 + 支払方法未登録)	trial_expired_no_payment_method
suspended	active	IF #1(支払再開 / 解除)	-
active / sus- pended	deleted_pending	admin 退会申請 (SCR-024)	-
deleted_pending	active / suspended(元の状 態)	admin 撤回 (30 日以内)	-
deleted_pending	deleted	30 日経過 + 物理削除 Batch	-

1.9 9. データ削除モード

設定: `contract_owners.data_deletion_mode(physical / anonymize、初期値 physical)` で保持する独立
 カラム。

論理削除 は `UPDATE ... SET valid=0` で行い、90 日経過後の物理削除バッチ (DD14) で物理削除モード
 (`physical / anonymize`) に従って実体を消去する。SCR-017-M1 のメンバー削除と SCR-026 のプロジェクト
 削除は常に論理削除を経由する。

1.9.1 9.1 削除モード対応表

対象テー ブル	論理削除	物理削除 (90 日後バッチ or 即時、 <code>data_deletion_mode='physical'</code>)	匿 名 化 (<code>data_deletion_mode='anonymize'</code>)
users	<code>valid=0</code>	DELETE(ON DELETE CASCADE で関連 行連鎖)	email_encrypted → anon_<sha256>, name → 「匿名」
contract_owners	<code>valid=0(contract_status='deleted' と必ず同時セット = CHECK)</code>	DELETE(users CASCADE 経由)	(匿名化対象外、users 連鎖物 理削除)
project_users	<code>valid=0</code>	DELETE(users または projects CAS- CADE 経由)	(匿名化対象外、CASCADE 連 鎖物理削除)
projects	<code>status='deleted', valid=0</code>	DELETE(関連 faqs / inquiries 等は CAS- CADE 経由)	name → 「削除済みプロジェ クト」
allowed_domains	<code>valid=0</code>	DELETE	(匿名化対象外)
project_ip_allowlist	<code>valid=0</code>	DELETE	(匿名化対象外)
faqs	<code>status='deleted', valid=0</code>	DELETE	title / body 空字
question_logs	<code>valid=0</code>	DELETE	user_question 空字, ip_address / user_agent NULL, 集計値維持
inquiries	<code>valid=0</code>	DELETE	user_question 空字, inquiry_code 維持, closed_reason 空字
inquiry_contacts	<code>valid=0</code>	DELETE	email_encrypted → anon_<sha256>, dis- play_name → 「匿名」
chat_rooms	<code>valid=0</code>	DELETE	(匿名化対象外、chat_messages 単位で匿名化)
chat_room_revisions	(valid なし、履歴系で論理削除 概念外)	chat_rooms 物理削除時に ON DELETE CASCADE で連鎖物理削除	(匿名化対象外)
chat_messages	(valid なし、 chat_rooms.valid=0 で十分)	DELETE	content 空字, actor_type 維持
billing_subscriptions	<code>valid=0</code>	(物理削除バッチ対象外、永久保持)	(匿名化対象外、電子帳簿保存 法)
notification_logs	(valid なし、ログ系で論理削除 概念外)	DELETE	recipient_email_hmac → SHA-256, vars → 空 JSON

対象テーブル	論理削除	物理削除 (90 日後バッチ or 即時、 data_deletion_mode='physical')	匿名化 (data_deletion_mode='anonymize')
announcements	(valid なし、email_encrypted 配信関係 M:N)	service_announcements / contract_owners 物理削除時に ON DELETE CASCADE で連鎖物理削除	(匿名化対象外)
access_tokens	(valid なし、used_at で代用)	DELETE	(常に物理削除、匿名化対象外)
sessions	(valid なし、revoked_at で代用)	DELETE	(匿名化対象外)
audit_logs	(valid なし、メイン側は API 操作ログ専用)	created_at + retention 期間 経過後に物理 DELETE (general 1y / billing 7y / operator_high_priv 5y)	(匿名化対象外。ハッシュチェーン正本は運営者側)

1.9.2 9.2 論理削除の物理削除バッチ (DD14 正本)

- ・ 日次バッチで `valid=0 AND updated_at < now() - 90d` の行を物理削除モード (`contract_owners.data_deletion_mode` 準拠) に従って処理
- ・ `users` 行が物理 DELETE された時点で **ON DELETE CASCADE** で関連 `contract_owners/project_users/sessions/access_tokens/terms_agreements` 等が連鎖物理削除
- ・ `projects` 行が物理 DELETE された時点で **ON DELETE CASCADE** で関連 `faqs/inquiries` 等が連鎖物理削除
- ・ `billing_subscriptions` は物理削除バッチの対象外 (永久保持)

1.10 10. データ保持の調整制約 (NFR-702)

- ・ データ保持期間はシステム固定値とし、契約・管理者ユーザーによる UI 上の変更は不可 (短縮・長期化いずれも不可)
- ・ 固定値は本書 §2 のテーブル一覧「保持期間」列および要件定義 NFR-701/NFR-705 を正本とする
- ・ 短縮が必要な場合 (法令・契約義務範囲内) はサポート窓口経由の運営者対応とし、長期化が必要な場合は将来要件 (FUT01_認証・セキュリティ強化/FUT 配下) で再検討する
- ・ SCR-028 アカウント設定にデータ保持期間設定 UI を持たない (基本設計 01_画面設計.md §5.20b 参照)
- ・ `usage_metering/billing_invoices/audit_logs(retention_class='billing')` は電子帳簿保存法・国税法の要請により最低 7 年保持 (NFR-602b)、これを下回る短縮は不可

1.11 11. PII 取扱 (§7.10 から移管)

観点	仕様
暗号化方式	AES-256-GCM 列単位、オーナー派生 (NFR-319)
対象テーブル	inquiry_contacts.email_encrypted、users.email_encrypted
検索方式	暗号化 + *_hmac カラム (HMAC-SHA256) で復号不要
ローテーション	年次 Master Key ローターション時に KDF 再実行。60 日 dual-decrypt 期間 (NFR-321)
派生入力	deriveOwnerKey(masterKey, contractOwnerUserId)(contractOwnerUserId = contract_owners.user_id = ID owner accounts.id と物理値同じ ULID)
カード情報	Stripe に委譲 (PCI-DSS 範囲外)

詳細は [10_セキュリティ設計書.md §3/§8](#) を正本とする。

1.12 12. インデックス方針 (§7.6 から移管)

観点	方針
FAQ 検索 (FTS5)	faq_search_fts 仮想テーブル、(project_id, status='published') 前段フィルタ
未解決質問一覧	(contract_owner_user_id, project_id, case_status, created_at DESC)
お知らせ一覧 (NFR-106)	(recipient_user_id, read_at)/(recipient_user_id, category, created_at DESC)
未読件数 (NFR-106)	KV キャッシュ inbox:unread:<user_id>(TTL 60 秒) + DB フォールバック
ユニーク	inquiry_code, widget_public_key, access_tokens.token_hash, (contract_owner_user_id, billing_year_month)

観点	方針
監査ログ	(contract_owner_user_id, action, created_at), (retention_class, created_at)(保持期間バッチ用)

1.13 13. 未決事項

No	内容	確認先	期限	ステータス
1	詳細設計書の inquiries.case_status に CHECK 制約欠落(本書では補正済)	詳細設計書 行 6060	-	補正待ち
2	rate_limit Overrides / budget_limit Overrides を別テーブルに分離するか(現状 owner_quota Overrides.resource_kind で統一)	-	-	既決: 統一実装で継続